

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 7

End-to-End Study of Autoencoders, Convolutional Autoencoders, Denoising Autoencoders and Variational Autoencoders

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

The objective of this experiment is to develop an end-to-end understanding of autoencoders and their variants for image representation, reconstruction, denoising and generative modeling.

The experiment begins with a fully connected autoencoder, progresses to a Convolutional Autoencoder (CAE), introduces image corruption for denoising, and finally develops a Variational Autoencoder (VAE). Students will visualize the learned latent representation, compare reconstruction quality using quantitative and qualitative measures, and explore the generative capability of the VAE.

The same image dataset and a controlled experimental protocol should be used throughout the experiment wherever possible.

2. Learning Outcomes

After completing this experiment, students will be able to:

- explain the encoder–latent–decoder structure of an autoencoder;
- prepare image data for reconstruction-based learning;
- implement a fully connected autoencoder;
- implement a Convolutional Autoencoder for image reconstruction;
- introduce controlled noise and build a denoising autoencoder;
- evaluate reconstruction using MSE, MAE and SSIM;
- visualize and interpret latent representations;
- explain the probabilistic latent representation of a VAE;
- implement the reparameterization trick;
- calculate and interpret reconstruction and KL-divergence losses;
- generate new images by sampling from a learned latent space; and
- compare deterministic and variational latent representations.

3. Dataset and Experimental Setup

Dataset: MNIST Handwritten Digit Dataset

MNIST contains grayscale images of handwritten digits from 0 to 9. Each image has spatial dimensions

$$28 \times 28 \times 1.$$

The dataset is particularly suitable for this experiment because the images are small, the classes have visually meaningful structure, and the same image format can be directly supplied to fully connected, convolutional and variational

autoencoders.

The dataset can be loaded directly using Keras/TensorFlow:

https://www.tensorflow.org/api_docs/python/tf/keras/datasets/mnist

Recommended laboratory subset:

Use 10,000 training images and 2,000 test images for the main laboratory execution. The full MNIST dataset may be used if computational resources permit.

The labels are not required as reconstruction targets. The original image is the target:

$$x \rightarrow \text{Encoder} \rightarrow z \rightarrow \text{Decoder} \rightarrow \hat{x}$$

where x is the input image and $\hat{x}$ is its reconstruction.

4. Expected Input and Output

Expected Input

Each input image has shape

$$28 \times 28 \times 1.$$

For a batch of 32 images:

$$X \in \mathbb{R}^{32 \times 28 \times 28 \times 1}.$$

Pixel values should be converted from

$$[0, 255] \rightarrow [0, 1].$$

For a fully connected autoencoder, the image may be flattened:

$$28 \times 28 = 784.$$

Therefore,

$$x \in \mathbb{R}^{784}.$$

For a convolutional autoencoder, retain the spatial structure:

$$x \in \mathbb{R}^{28 \times 28 \times 1}.$$

Expected Output

The primary output is a reconstructed image:

$$\hat{x} \in \mathbb{R}^{28 \times 28 \times 1}.$$

Students must display:

Original image

Reconstructed image

Reconstruction error

For the denoising model, display:

Original clean image

Noisy image

Denoised reconstruction

For the VAE, display newly generated samples obtained by sampling from the latent distribution.

5. Overall Experimental Pipeline

The complete experiment follows:



The experiment consists of four related studies:

1. Fully Connected Autoencoder
2. Convolutional Autoencoder
3. Denoising Convolutional Autoencoder
4. Variational Autoencoder

6. Exploring Autoencoders

An autoencoder learns to reproduce its input at the output.

The encoder maps the input to a lower-dimensional representation:

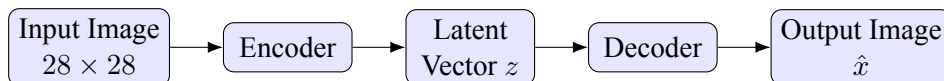
$$z = f_{\theta}(x),$$

and the decoder reconstructs the input:

$$\hat{x} = g_{\phi}(z).$$

The training objective is to minimize reconstruction error:

$$\mathcal{L}_{rec} = \frac{1}{N} \sum_{i=1}^N \|x_i - \hat{x}_i\|^2.$$



The latent vector is a compressed representation of the input.

7. Fully Connected Autoencoder

For the first implementation, a fully connected autoencoder was used to reconstruct the MNIST images. Since a fully connected layer expects a one-dimensional input, each 28×28 grayscale image was first flattened into a vector of length

$$28 \times 28 = 784. \quad (1)$$

The encoder gradually reduced this 784-dimensional input to a latent representation of size 16. The decoder then expanded the latent representation back to the original dimension of 784.

The architecture used was

$$784 \rightarrow 128 \rightarrow 32 \rightarrow 16 \rightarrow 32 \rightarrow 128 \rightarrow 784. \quad (2)$$

The 16-dimensional layer was treated as the bottleneck or latent representation. ReLU activation was used in the hidden layers and sigmoid activation was used in the final layer so that the reconstructed pixel values remained between 0 and 1.

7.1 Network Architecture

The complete architecture of the fully connected autoencoder is shown in Figure 1.

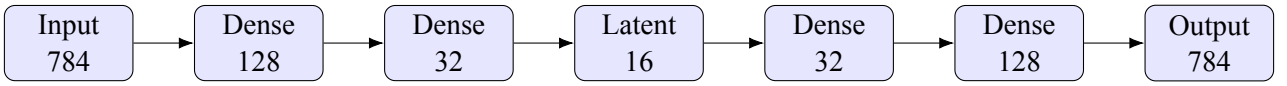


Figure 1: Architecture of the fully connected autoencoder.

The input layer receives the flattened image containing 784 pixel values. The encoder consists of two dense layers with 128 and 32 neurons, followed by the 16-neuron latent layer. The decoder mirrors this reduction by using dense layers with 32 and 128 neurons before producing the final 784-dimensional reconstruction.

7.2 Training Configuration

The model was trained using the Adam optimizer with a learning rate of 10^{-3} . A batch size of 128 was used and the model was trained for 20 epochs. Binary cross-entropy was used as the reconstruction loss.

Table 1: Training configuration of the fully connected autoencoder.

Parameter	Value
Input dimension	784
Latent dimension	16
Hidden-layer activation	ReLU
Output activation	Sigmoid
Optimizer	Adam
Learning rate	10^{-3}
Batch size	128
Number of epochs	20
Loss function	Binary Cross-Entropy

The same normalized MNIST images were used as both the input and the target. Therefore, the network was trained to minimize the difference between the original image and its reconstruction rather than to predict the digit label.

8. Required Analysis of the Basic Autoencoder

8.1 Original versus Reconstructed Images

The first qualitative evaluation of the fully connected autoencoder was performed by comparing the original MNIST test images with their reconstructed images. Ten test samples were selected for the comparison. The original images are shown in the first row of Figure 2, while the corresponding reconstructed images are shown in the second row.

FC Autoencoder: Original vs Reconstruction



Figure 2: Comparison of original MNIST images with their reconstructions obtained using the fully connected autoencoder.

From the figure, most of the digits are reconstructed with their main shape preserved, and they are still easily recognizable. Digits such as 7, 2, 1, 0 and 4 retain their basic stroke pattern.

The reconstructed images are slightly smoother than the originals. Some thin strokes become softer, and small variations in the handwritten strokes are lost, which is more noticeable in digits like 2, 6, 8 and 9. The digit 1 is reconstructed more easily because of its simple shape.

This loss of detail is mainly due to the compression of the 784 input values into only 16 latent values. Overall, the autoencoder captures the main structure of the digits well, but some fine pixel-level details are lost during reconstruction.

8.2 Training and Validation Reconstruction Loss

The training and validation reconstruction losses were recorded during the 20 training epochs to observe how the fully connected autoencoder learned to reconstruct the MNIST images. The corresponding curves are shown in Figure 3.

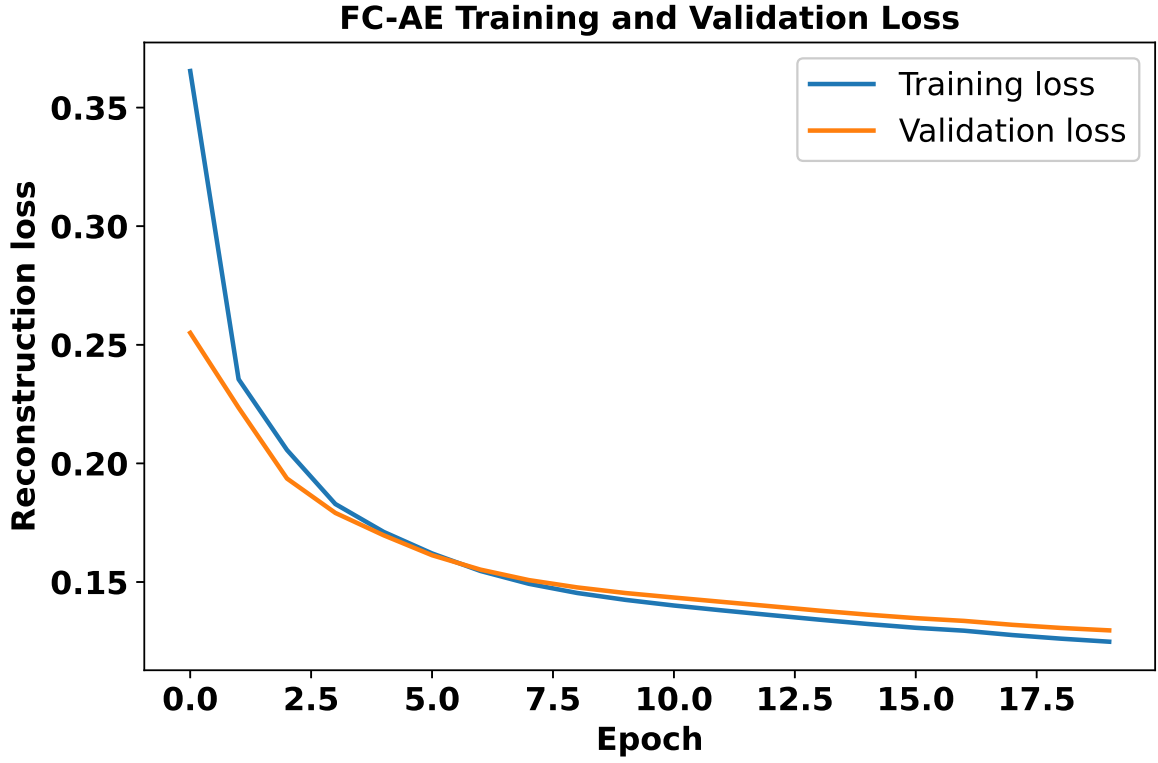


Figure 3: Training and validation reconstruction loss of the fully connected autoencoder.

Both curves show a clear decrease in reconstruction loss during the initial epochs, which indicates that the model was learning to reconstruct the input images. The decrease is more noticeable in the earlier epochs, while the changes become smaller towards the end of training.

The validation loss follows a similar trend to the training loss and does not show a continuous increase while the training loss decreases. Also, the two curves do not show a large gap in the later epochs, so there is no strong evidence of overfitting within the 20 epochs.

Overall, the model learns the main features of the digits fairly quickly, while the later epochs produce only small improvements. The curves therefore suggest that the model had largely converged by the end of training.

9.Reconstruction Metrics

Visual inspection gives an initial idea of how well the autoencoder reconstructs the input images, but numerical measures are required to compare the reconstruction more objectively. Therefore, the reconstructed images of the held-out test set were evaluated using Mean Squared Error (MSE), Mean Absolute Error (MAE), and Structural Similarity Index (SSIM).

9.1 Mean Squared Error

Mean Squared Error measures the average squared difference between the original and reconstructed pixel values. It is given by

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (x_i - \hat{x}_i)^2, \quad (3)$$

where x_i represents the original pixel value and $\hat{x}_i$ represents the corresponding reconstructed pixel value.

For the fully connected autoencoder, the obtained test MSE was

$$\boxed{\text{MSE} = 0.022480}. \quad (4)$$

9.2 Mean Absolute Error

Mean Absolute Error measures the average absolute difference between the original and reconstructed pixel values:

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |x_i - \hat{x}_i|. \quad (5)$$

The test-set MAE obtained from the experiment was

$$\boxed{\text{MAE} = 0.059559}. \quad (6)$$

9.3 Structural Similarity Index

SSIM was used to measure the similarity between the original and reconstructed images while considering their local structural information. The mean SSIM calculated over the test images was

$$\boxed{\text{SSIM} = 0.730615}. \quad (7)$$

9.4 Results

The three metrics obtained from the fully connected autoencoder are summarized in Table 2.

Table 2: Test-set reconstruction metrics for the fully connected autoencoder.

Metric	FC-AE Value
Test MSE	0.022480
Test MAE	0.059559
Mean SSIM	0.730615

The numerical results support the observations from the reconstructed images. The MSE of 0.022480 and MAE of 0.059559 show that there is still some difference between the original and reconstructed pixels, which can also be seen as smoothing in some of the digit strokes.

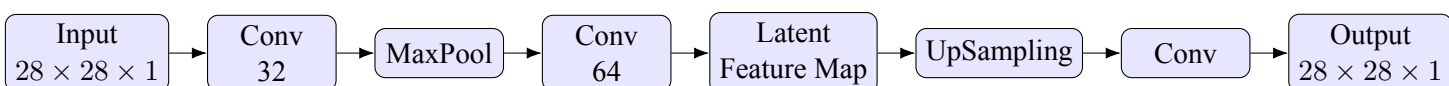
The SSIM value of 0.730615 shows that the reconstructed images retain a good amount of the original structure. However, since the value is not close to 1, some fine stroke details and sharper boundaries are still lost.

Overall, the results show that the fully connected autoencoder captures the general structure of the MNIST digits, but compressing the images into a 16-dimensional latent space causes some loss of pixel-level detail.

10. Convolutional Autoencoder

A Convolutional Autoencoder preserves spatial structure and uses convolutional operations in the encoder and up-sampling/transpose convolution operations in the decoder.

The conceptual architecture is:



10.1 CAE Architecture

The convolutional encoder uses three convolutional layers with 32, 64 and 64 filters. Max-pooling is applied after the first two convolutional layers to reduce the spatial dimensions while extracting features.

The decoder uses upsampling followed by convolutional layers to restore the original image size. The final convolutional layer has one filter with sigmoid activation, which produces the reconstructed grayscale image.

The architecture used in the experiment was:

Layer	Output / Operation
Input	$28 \times 28 \times 1$
Conv2D	32 filters, 3×3 , ReLU
MaxPooling2D	2×2
Conv2D	64 filters, 3×3 , ReLU
MaxPooling2D	2×2
Conv2D	64 filters, 3×3 , ReLU
UpSampling2D	2×2
Conv2D	32 filters, 3×3 , ReLU
UpSampling2D	2×2
Conv2D	1 filter, 3×3 , Sigmoid
Output	$28 \times 28 \times 1$

The main change from the fully connected autoencoder is that the image is not flattened before feature extraction. The convolutional layers operate on local regions of the image, allowing neighbouring pixels to contribute to the learned features.

10.2 Encoder

The encoder first applies a convolution with 32 filters while maintaining the 28×28 spatial dimensions. A 2×2 max-pooling operation then reduces the spatial size to 14×14 .

The second convolution increases the number of feature maps to 64, followed by another max-pooling operation. The spatial dimensions are consequently reduced to 7×7 .

The third convolution operates on the resulting $7 \times 7 \times 64$ feature maps. These feature maps form the compact spatial representation used by the decoder.

The dimensional changes through the encoder can be summarized as

$$28 \times 28 \times 1 \rightarrow 28 \times 28 \times 32 \rightarrow 14 \times 14 \times 32 \rightarrow 14 \times 14 \times 64 \rightarrow 7 \times 7 \times 64 \rightarrow 7 \times 7 \times 64. \quad (8)$$

10.3 Decoder

The decoder starts from the $7 \times 7 \times 64$ feature representation. Upsampling is first used to increase the spatial dimensions to 14×14 . A convolutional layer with 32 filters then processes the upsampled feature maps.

A second upsampling operation restores the spatial dimensions to 28×28 . Finally, a convolutional layer with one filter and sigmoid activation produces the reconstructed grayscale image.

Thus, the decoder restores the original spatial dimensions:

$$7 \times 7 \times 64 \rightarrow 14 \times 14 \times 64 \rightarrow 14 \times 14 \times 32 \rightarrow 28 \times 28 \times 32 \rightarrow 28 \times 28 \times 1. \quad (9)$$

10.4 Training Configuration

The CAE was trained using the same normalized MNIST images as both the input and reconstruction target. Adam optimization with a learning rate of 10^{-3} was used, with a batch size of 128 and 20 training epochs. Binary cross-entropy was used as the reconstruction loss.

Table 3: Training configuration of the convolutional autoencoder.

Parameter	Value
Input shape	$28 \times 28 \times 1$
Encoder filters	32, 64, 64
Decoder filters	64, 32, 1
Kernel size	3×3
Pooling	2×2 MaxPooling
Upsampling	2×2
Hidden activation	ReLU
Output activation	Sigmoid
Optimizer	Adam
Learning rate	10^{-3}
Batch size	128
Number of epochs	20
Loss function	Binary Cross-Entropy

Unlike the fully connected autoencoder, the CAE retains the two-dimensional arrangement of the image throughout most of the encoding and decoding process. The effect of this architectural difference will be examined using the reconstruction comparison in the next task.

11. Comparison of Fully Connected and Convolutional Autoencoders

The fully connected autoencoder and convolutional autoencoder were evaluated using the same MNIST test images. This makes the comparison more meaningful because the difference in reconstruction quality is mainly associated with the architecture used by the two models.

11.1 Quantitative Comparison

The test-set reconstruction results obtained from both models are shown in Table 4.

Table 4: Comparison of fully connected and convolutional autoencoders.

Metric	FC-AE	CAE
Test MSE	0.022480	0.002697
Test MAE	0.059559	0.015370
Mean SSIM	0.730615	0.973007

The difference between the two models can be seen clearly in all three metrics. The MSE decreases from 0.022480 for the fully connected autoencoder to 0.002697 for the convolutional autoencoder, showing a much smaller pixel-level error for the CAE.

The MAE also decreases from 0.059559 to 0.015370, meaning that the average difference between the original and reconstructed pixels is lower for the CAE.

The SSIM shows an even larger difference. The FC-AE obtained a mean SSIM of 0.730615, while the CAE obtained 0.973007. Since SSIM measures structural similarity, this higher value is consistent with the better preservation of the digit shapes in the convolutional reconstructions.

Overall, the CAE gives better reconstruction quality in this experiment, which can be related to its ability to preserve spatial information through convolutional layers.

11.2 Reconstruction Comparison

Figure 4 compares the original image with the reconstructions produced by the two models.

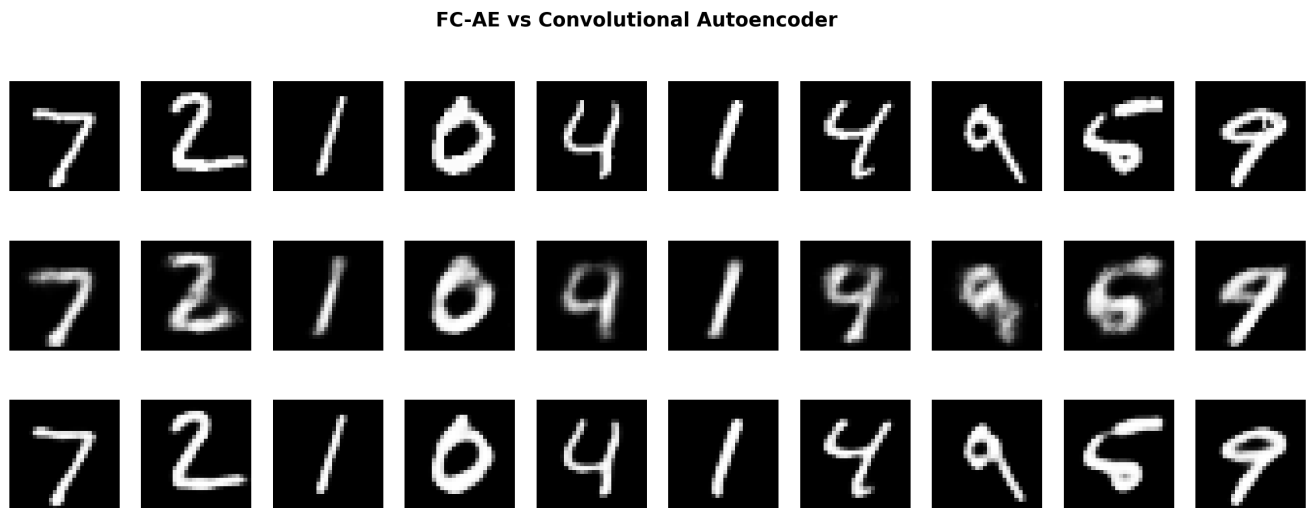


Figure 4: Reconstruction comparison between the fully connected autoencoder and convolutional autoencoder.

The visual difference between the two reconstructions is consistent with the quantitative results. The FC-AE preserves the general identity of the digits, but some strokes appear smoother and less defined. In comparison, the CAE preserves the digit shapes and boundaries more closely to the original images.

This difference comes from how the two models process the input. The FC-AE flattens the image into a 784-dimensional vector, so the spatial relationship between neighbouring pixels is not directly preserved. The CAE, on the other hand, uses convolutional filters over local regions, allowing it to retain spatial information while extracting features.

This is particularly useful for MNIST images because the position and shape of the digit strokes are important. The numerical results also support this observation: MSE decreases from 0.022480 to 0.002697, MAE decreases from 0.059559 to 0.015370, and SSIM increases from 0.730615 to 0.973007. Thus, in this experiment, preserving spatial information through convolution resulted in better reconstruction quality.

11.3 Interpretation

The comparison shows that both models learned the general appearance of the MNIST digits, but the CAE reconstructed the images more closely. This is also reflected in all three metrics, with MSE and MAE decreasing while SSIM increased considerably.

The increase in SSIM from 0.730615 to 0.973007 agrees with the visual observation that the CAE preserves the digit strokes and boundaries more clearly. Therefore, for the MNIST images used in this experiment, preserving spatial locality had a clear effect on the reconstruction quality.

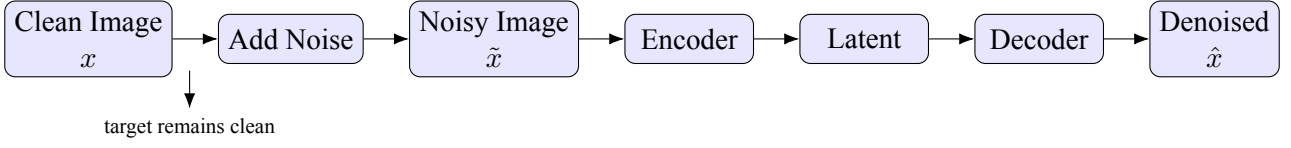
12. Denoising Autoencoder

A denoising autoencoder receives a corrupted image but is trained to reconstruct the original clean image.

The training mapping is:

$$\tilde{x} \rightarrow \text{Encoder} \rightarrow z \rightarrow \text{Decoder} \rightarrow \hat{x}$$

where $\tilde{x}$ is a noisy version of the clean image x .



The important difference from an ordinary autoencoder is that the target remains the original clean image. Therefore, the model is trained to remove the corruption rather than reproduce it.

12.1 Types of Noise Used

Two different types of corruption were considered in the experiment:

1. Gaussian noise
2. Salt-and-pepper noise

Using two different corruption mechanisms allows the denoising model to be examined under different types of disturbances.

12.2 Gaussian Noise

Gaussian noise was generated by adding a random value sampled from a normal distribution to each pixel:

$$\tilde{x} = x + n, \quad (10)$$

where

$$n \sim \mathcal{N}(0, \sigma^2). \quad (11)$$

Three noise levels were used:

Gaussian Noise Level
$\sigma = 0.1$
$\sigma = 0.2$
$\sigma = 0.3$

After adding the noise, the resulting pixel values were clipped to the valid range $[0, 1]$.

$$\tilde{x} = \text{clip}(x + n, 0, 1). \quad (12)$$

Increasing σ increases the amount of random variation introduced into the input image. Therefore, the model has to remove progressively stronger corruption while reconstructing the same clean target.

12.3 Salt-and-Pepper Noise

Salt-and-pepper noise was also introduced by randomly replacing a proportion of the image pixels with extreme values. The corruption probabilities used were

Salt-and-Pepper Noise Level
$p = 0.05$
$p = 0.10$
$p = 0.20$

For a pixel selected for corruption, the value was changed to either 0 or 1. Pixels that were not selected retained their original values.

Thus, the two noise types affect the input differently. Gaussian noise introduces continuous pixel-level variations, whereas salt-and-pepper noise produces isolated extreme-valued pixels.

12.4 Denoising CAE Architecture

The denoising model was based on the convolutional autoencoder architecture used earlier. The main difference was the training input.

For the ordinary CAE, the training pair was

$$(x, x), \quad (13)$$

whereas for the denoising CAE it became

$$(\tilde{x}, x). \quad (14)$$

Therefore, the model receives the noisy image but learns to produce the clean image.

The convolutional structure was retained so that the network could use local spatial information while learning to reconstruct the digit. The output layer used sigmoid activation so that the reconstructed image remained in the range $[0, 1]$.

12.5 Training Procedure

For each noisy training sample, the corresponding clean image was used as the target. The same basic reconstruction objective was therefore used:

$$L_{\text{rec}} = L(\hat{x}, x), \quad (15)$$

where x is the clean image and $\hat{x}$ is the denoised reconstruction.

The denoising model was evaluated using the same MSE, MAE and SSIM measures used for the earlier reconstruction experiments. This allowed the effect of increasing noise levels to be studied quantitatively in addition to the visual comparison of clean, noisy and denoised images.

13. Adding Image Noise

Use at least two corruption types.

Gaussian Noise

$$\tilde{x} = x + n, \quad n \sim \mathcal{N}(0, \sigma^2).$$

Suggested standard deviations:

$$\sigma \in \{0.1, 0.2, 0.3\}.$$

Clip the resulting image to the range $[0, 1]$.

Salt-and-Pepper Noise

Randomly replace a small proportion of pixels with values close to 0 or 1.

Suggested corruption levels:

$$p \in \{0.05, 0.10, 0.20\}.$$

14. Required Denoising Plots

14.1 Clean, Noisy and Denoised Images

To visually evaluate the denoising autoencoder, the same MNIST images were considered in three forms: the original clean image, the image after adding noise, and the reconstructed image obtained from the denoising autoencoder. The model was trained using the noisy image as the input while the corresponding clean image was used as the target.

Figure 5 shows the comparison for representative test images.

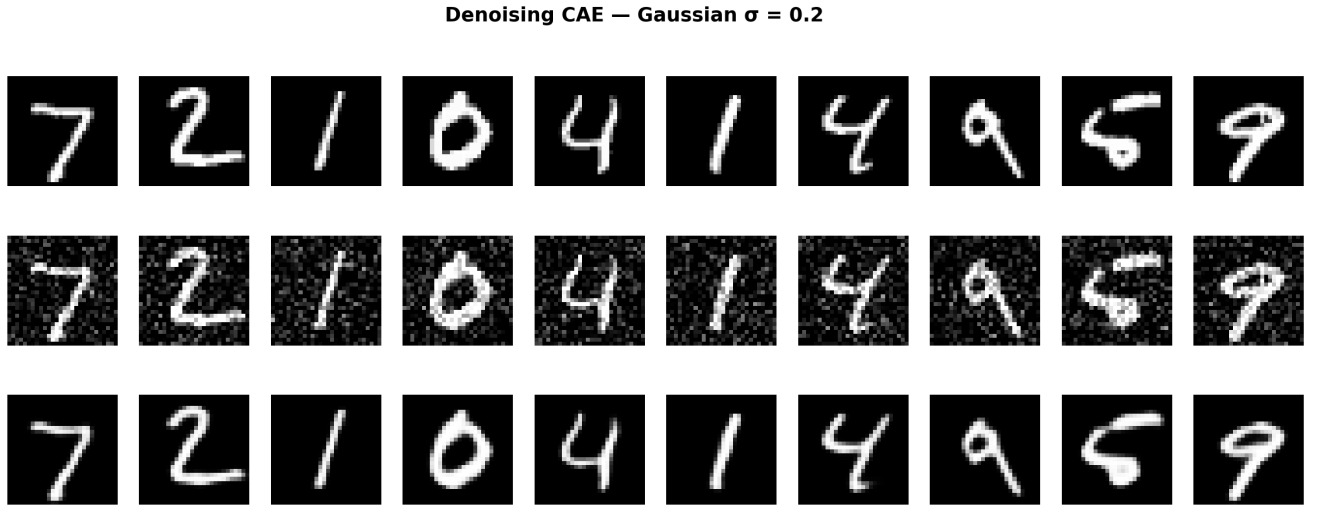


Figure 5: Comparison of clean, noisy and denoised MNIST images.

Interpretation

The clean images are used as the reference to compare the denoising results. After adding noise, the digits become less clear because random changes are introduced into the pixel values. The denoised images show that the autoencoder is able to recover the main shape of the digits from the noisy inputs.

Most of the main digit structure is preserved, while a good amount of the added noise is removed. Some fine details are slightly smoothed during reconstruction. This is expected because the model learns the common structure of the digits instead of trying to reproduce every noisy pixel.

It can also be seen that denoising becomes more difficult as the noise level increases. At higher noise levels, some small stroke details are lost, but the overall digit shapes remain recognizable. This is further analyzed using MSE, MAE and SSIM for the different noise levels (Table 5).

14.2 Noise Level vs. Reconstruction Quality

The denoising autoencoder was evaluated using three levels of Gaussian noise, $\sigma = 0.1$, $\sigma = 0.2$, and $\sigma = 0.3$. The reconstructed images were compared with the corresponding clean images using MSE, MAE, and SSIM.

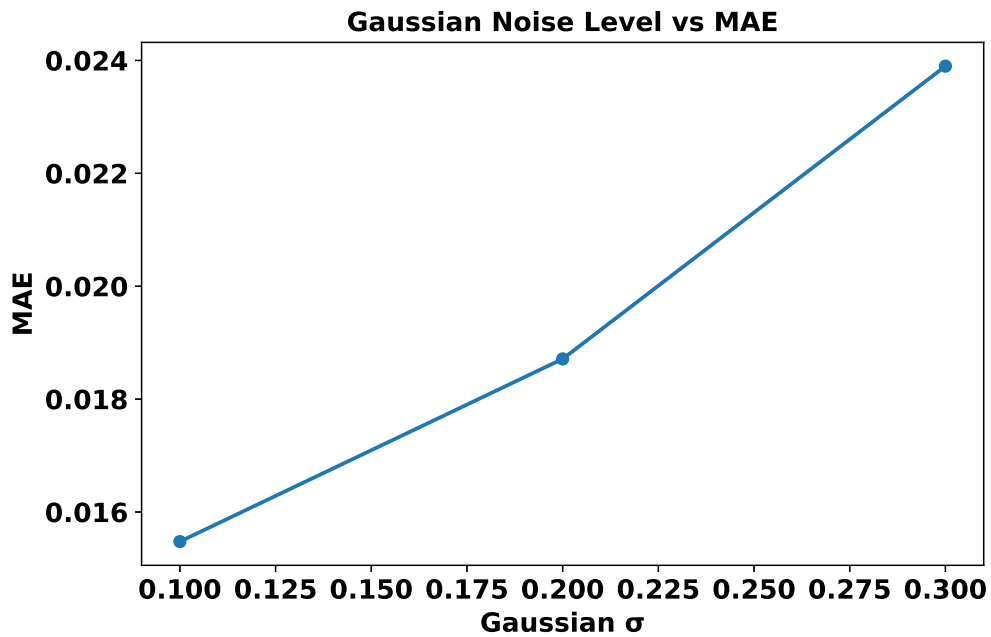


Figure 6: Gaussian noise level versus MAE.

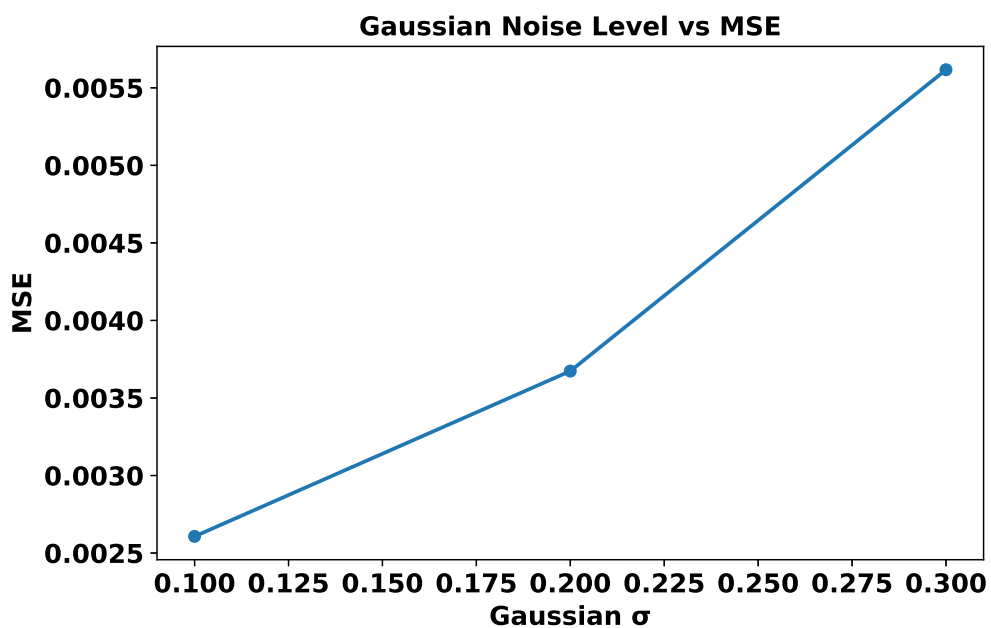


Figure 7: Gaussian noise level versus MSE.

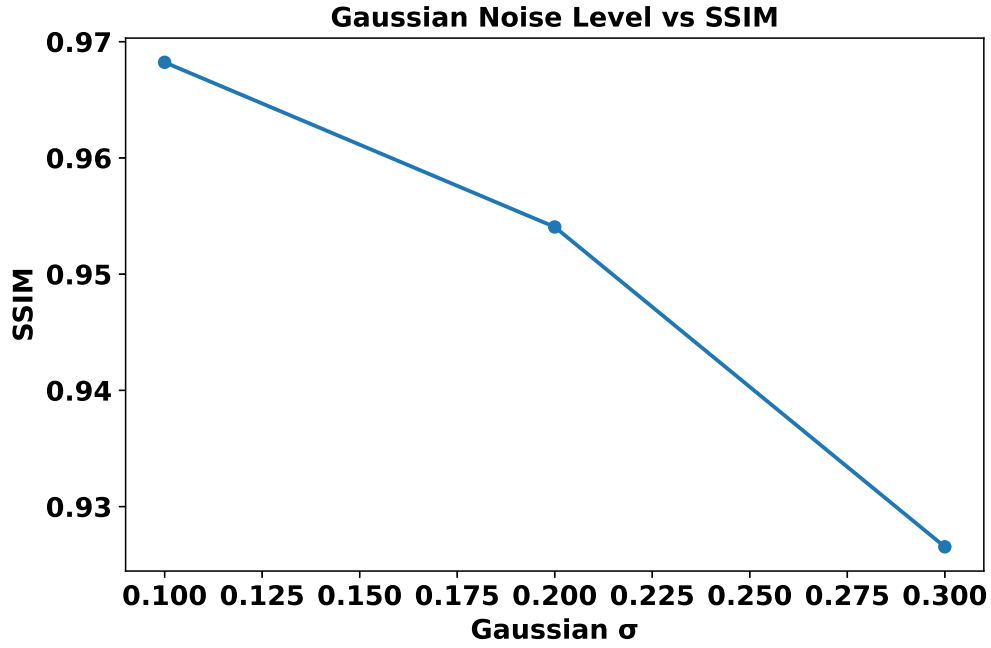


Figure 8: Gaussian noise level versus SSIM.

The numerical results obtained at the three noise levels are given in Table 5.

Table 5: Denoising performance for different Gaussian noise levels.

Noise Level (σ)	MSE	MAE	SSIM
0.1	0.002608	0.015477	0.968226
0.2	0.003673	0.018712	0.954060
0.3	0.005617	0.023899	0.926540

Interpretation

As the Gaussian noise level increases, the reconstruction performance decreases. The MAE increases from 0.015477 to 0.023899 and the MSE increases from 0.002608 to 0.005617 as σ changes from 0.1 to 0.3.

At the same time, SSIM decreases from 0.968226 to 0.926540, showing that the reconstructed images become less structurally similar to the clean images at higher noise levels. Overall, stronger noise makes denoising more difficult, although the main image structure is still preserved.

15. Variational Autoencoder

A conventional autoencoder maps an input to a deterministic latent vector:

$$z = f(x).$$

A VAE instead learns a probability distribution:

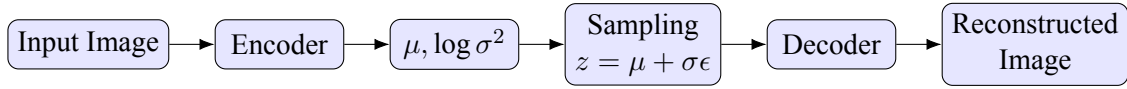
$$q_{\phi}(z|x) = \mathcal{N}(\mu(x), \text{diag}(\sigma^2(x))).$$

The encoder produces:

$$\mu, \quad \log \sigma^2.$$

A latent vector is sampled using the reparameterization trick:

$$z = \mu + \sigma \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I).$$



16. VAE Loss Function

The VAE objective consists of two components.

Reconstruction Loss

$$\mathcal{L}_{rec} = -\mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x|z)].$$

For normalized MNIST images, binary cross-entropy can be used.

KL Divergence

$$D_{KL}(q_\phi(z|x) \parallel p(z)) = -\frac{1}{2} \sum_j (1 + \log \sigma_j^2 - \mu_j^2 - \sigma_j^2).$$

The total loss is

$$\boxed{\mathcal{L}_{VAE} = \mathcal{L}_{rec} + \mathcal{L}_{KL}}$$

where the prior is usually

$$p(z) = \mathcal{N}(0, I).$$

17. VAE Implementation

Use a small latent space so that it can be visualized.

Suggested latent dimension:

$$z \in \mathbb{R}^2.$$

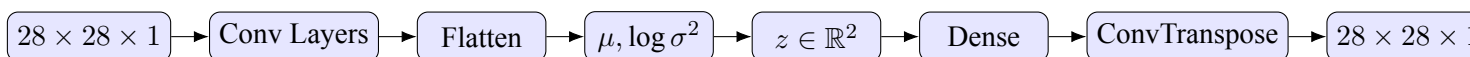
The encoder should produce:

$$\mu \in \mathbb{R}^2, \quad \log \sigma^2 \in \mathbb{R}^2.$$

The decoder maps

$$z \rightarrow 28 \times 28 \times 1.$$

A compact architecture is:



18. Exploring the VAE Latent Space

Since the VAE was designed with a two-dimensional latent space, the encoded test images can be visualized directly using the two latent variables. For each test image, the encoder produces a latent representation (z_1, z_2) . These points are plotted on a two-dimensional plane, with the digit labels used only to distinguish the different digit groups visually.

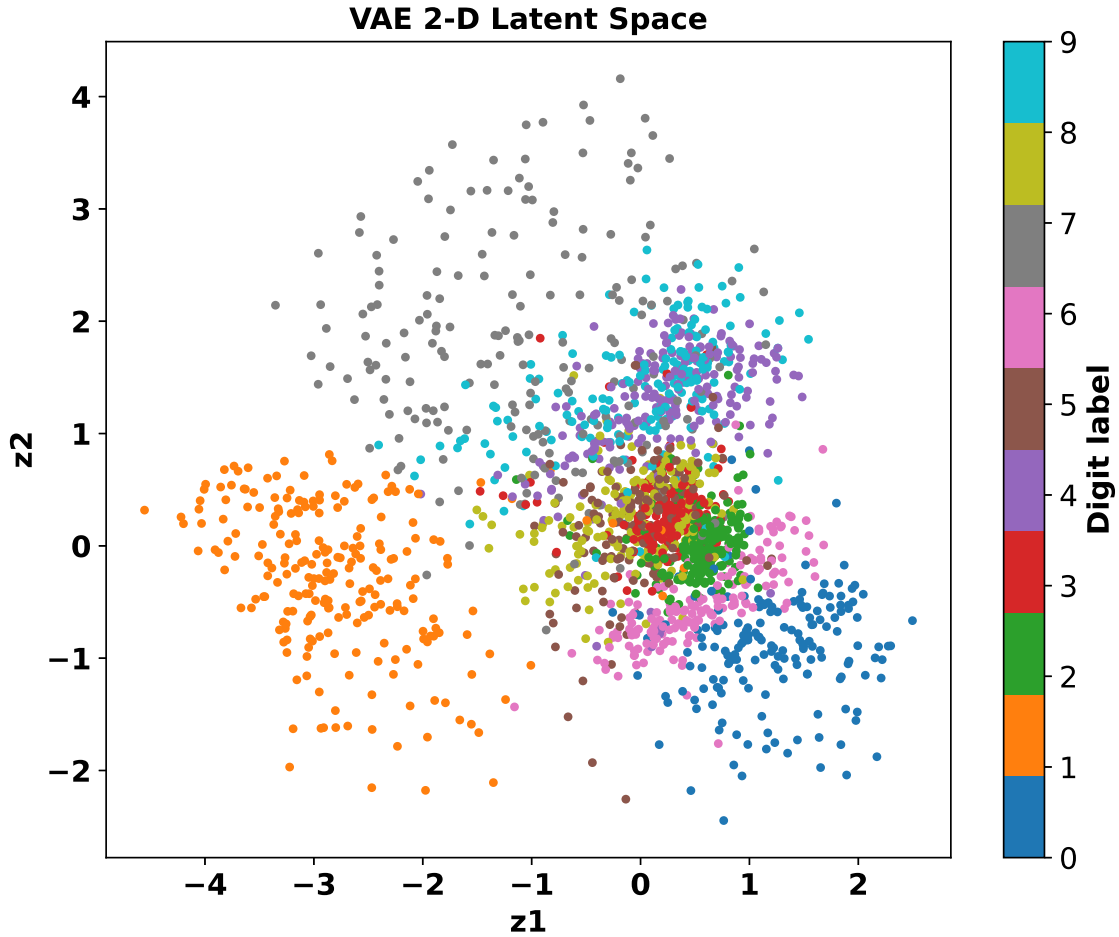


Figure 9: Two-dimensional latent-space representation learned by the VAE.

Interpretation

The plot shows how the MNIST test images are distributed in the learned two-dimensional latent space. Images of the same digit generally appear in nearby regions, although some overlap can be seen between different digit groups.

This is expected because the VAE learns a continuous probabilistic representation rather than explicitly separating the digits into classes. The latent space therefore captures some meaningful features of the digits, while still allowing different digit groups to overlap.

This continuous representation is also useful for generation, since points from the latent space can be passed through the decoder to generate new images. It is used in the following experiments for random image generation and latent-space interpolation.

19. Generating New Images with the VAE

Random Image Generation using VAE

One of the main advantages of a Variational Autoencoder is that it can generate new samples from its learned latent space. After training, latent vectors were randomly sampled from the standard normal distribution

$$z \sim \mathcal{N}(0, I). \quad (16)$$

The sampled latent vectors were given to the decoder of the trained VAE. The decoder then generated images of size $28 \times 28 \times 1$.

At least 25 latent vectors were sampled and decoded to obtain a grid of newly generated MNIST-like images.

25 Randomly Generated VAE Images

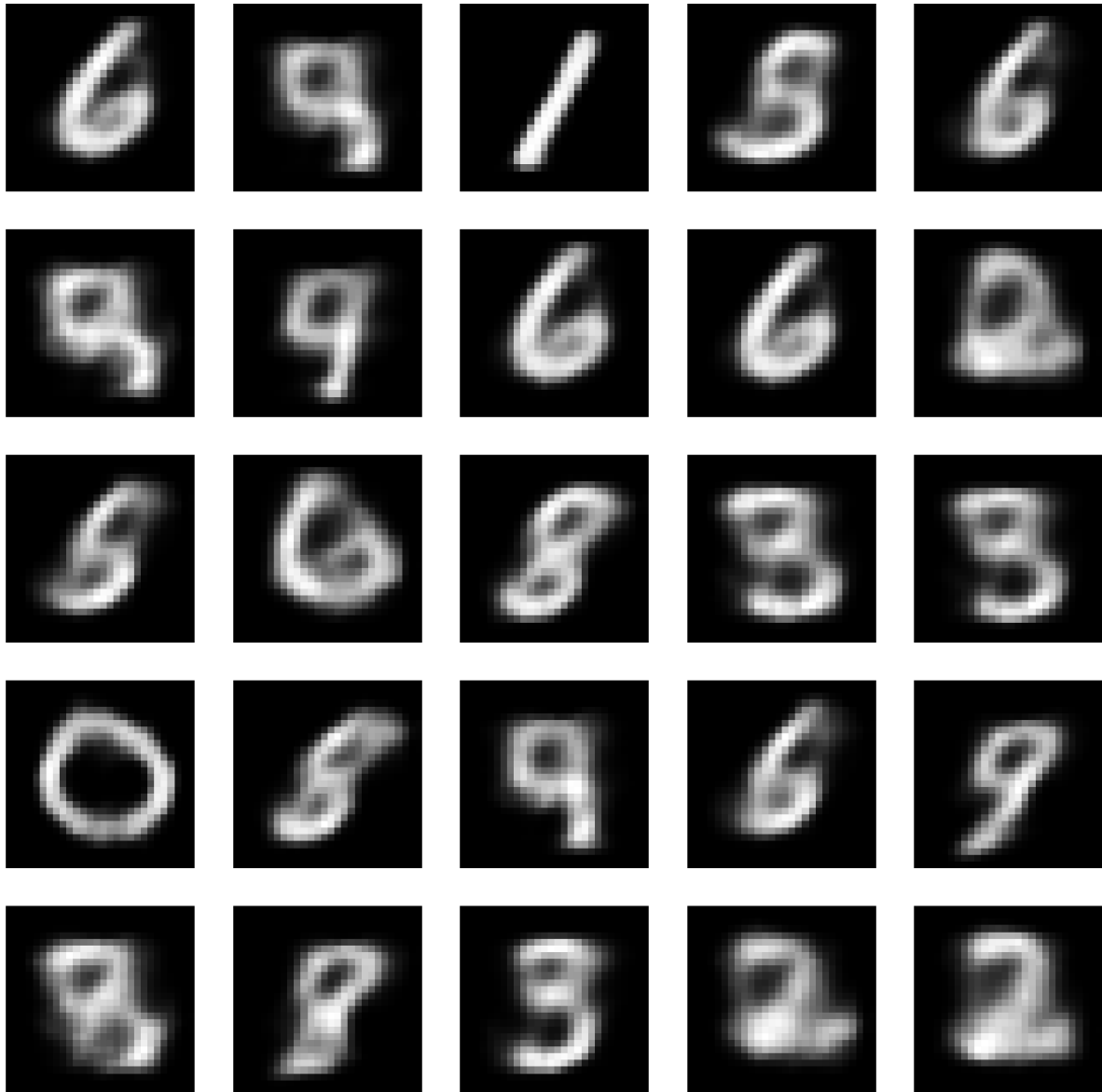


Figure 10: Randomly generated MNIST images obtained by sampling from the VAE latent space.

Interpretation

The generated images show the handwritten digit patterns learned by the VAE. Several samples have recognizable digit-like shapes, showing that the decoder has learned useful features from the MNIST images.

There is also some variation between the samples because the latent vectors are randomly sampled. A few images appear less clear or are difficult to identify as a single digit, which can happen when the sampled point lies between different digit regions in the latent space.

Overall, the generated-image grid shows that the VAE can produce new images from randomly sampled latent representations rather than only reconstructing the input images.

20. Latent Space Interpolation

The continuous nature of the VAE latent space was further examined using latent-space interpolation. Two latent vectors, z_A and z_B , were selected from the learned latent space. Intermediate latent vectors were then generated using

$$z(\alpha) = (1 - \alpha)z_A + \alpha z_B, \quad \alpha \in [0, 1]. \quad (17)$$

Values of α from 0 to 1 were used to move gradually from the first latent vector to the second. Each interpolated latent vector was passed through the trained decoder to obtain the corresponding generated image.

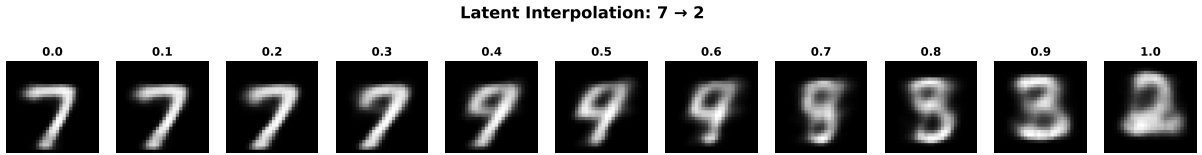


Figure 11: Latent-space interpolation between two VAE latent representations.

Interpretation

The interpolation sequence shows how the decoded image changes as the latent representation moves from z_A to z_B . The first image corresponds to $\alpha = 0$ and the final image corresponds to $\alpha = 1$, while the intermediate images are generated from points between them.

The images change gradually along the interpolation path instead of being generated independently. This shows the continuous nature of the VAE latent space, where the features of the decoded digit can change smoothly as the latent point moves between the two selected representations.

Overall, this experiment shows that the VAE can generate meaningful outputs from intermediate points in the learned latent space, rather than only from latent representations of the original training images.

21. Reconstruction Comparison Across Models

The reconstruction performance of the different autoencoder models was compared using the test set. The fully connected autoencoder (FC-AE), convolutional autoencoder (CAE), and denoising convolutional autoencoder were evaluated using MSE, MAE, and SSIM.

For the VAE, the reconstruction performance was also measured using the same image-based metrics. However, unlike the conventional autoencoders, the VAE is trained using a combined objective containing both reconstruction loss and KL-divergence loss. Therefore, these two components are considered separately when analysing the VAE.

Table 6: Comparison of reconstruction performance across autoencoder models

Model	MSE	MAE	SSIM
Fully Connected AE	0.022480	0.059559	0.730615
Convolutional AE	0.002697	0.015370	0.973007
Denoising CAE	0.003673	0.018712	0.954060
VAE	0.044133	0.103115	0.491307

Comparison of Reconstruction Results

The fully connected autoencoder obtained an MSE of 0.022480, MAE of 0.059559, and SSIM of 0.730615. In comparison, the convolutional autoencoder achieved lower reconstruction errors, with an MSE of 0.002697, MAE of 0.015370, and SSIM of 0.973007.

The denoising CAE also produced good reconstruction results, with an MSE of 0.003673, MAE of 0.018712, and SSIM of 0.954060. This shows that it was able to reconstruct the clean images even when Gaussian noise was added to the input. The VAE obtained an MSE of 0.044133, MAE of 0.103115, and SSIM of 0.491307. Its reconstruction quality is lower because the VAE also regularizes the latent space using KL-divergence.

Overall, the results show that the architecture and objective affect reconstruction quality. The CAE preserves the spatial structure of the MNIST images well, while the denoising CAE can recover the main image structure from noisy inputs. The VAE, although having higher reconstruction error, provides a probabilistic latent representation that is useful for generation and interpolation.

Table 7: VAE Loss Components and Reconstruction Metrics

Metric	Value
Reconstruction Loss	149.53836
KL Loss	5.90471
Total Loss	155.44308
Test MSE	0.044133
Test MAE	0.103115
Mean SSIM	0.491307

22. Reconstruction Error Distribution

To further analyse the reconstruction performance, the reconstruction error was calculated separately for each test image. For an image containing D pixels, the reconstruction error was calculated as the mean squared difference between the original and reconstructed pixels:

$$e_i = \frac{1}{D} \sum_{j=1}^D (x_{ij} - \hat{x}_{ij})^2. \quad (18)$$

The resulting error values were calculated over the test set and plotted as a histogram to observe the distribution of reconstruction errors.

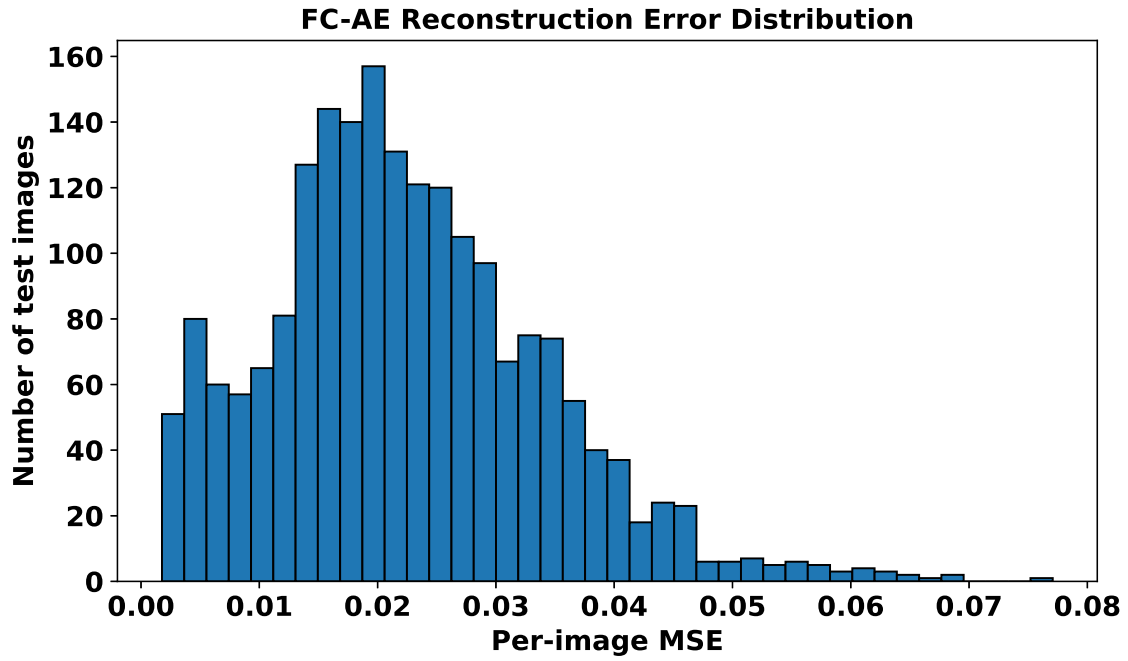


Figure 12: Distribution of reconstruction errors over the test set.

Interpretation

The histogram shows how the reconstruction error is distributed across the test images. Most images fall in the lower-error region, while a smaller number of images have comparatively higher errors.

Images with lower errors are reconstructed more closely by the autoencoder, whereas images with higher errors have a larger difference between the original and reconstructed outputs. The histogram is useful because the average MSE alone does not show how the error is distributed across individual images.

The higher-error samples can therefore be examined separately to identify the types of images that are more difficult for the model to reconstruct.

23. Required High-Error Image Analysis

The five test images with the largest reconstruction errors were selected from the test set. The original images and their corresponding reconstructed images were compared to identify the visual characteristics that contributed to the higher reconstruction error.

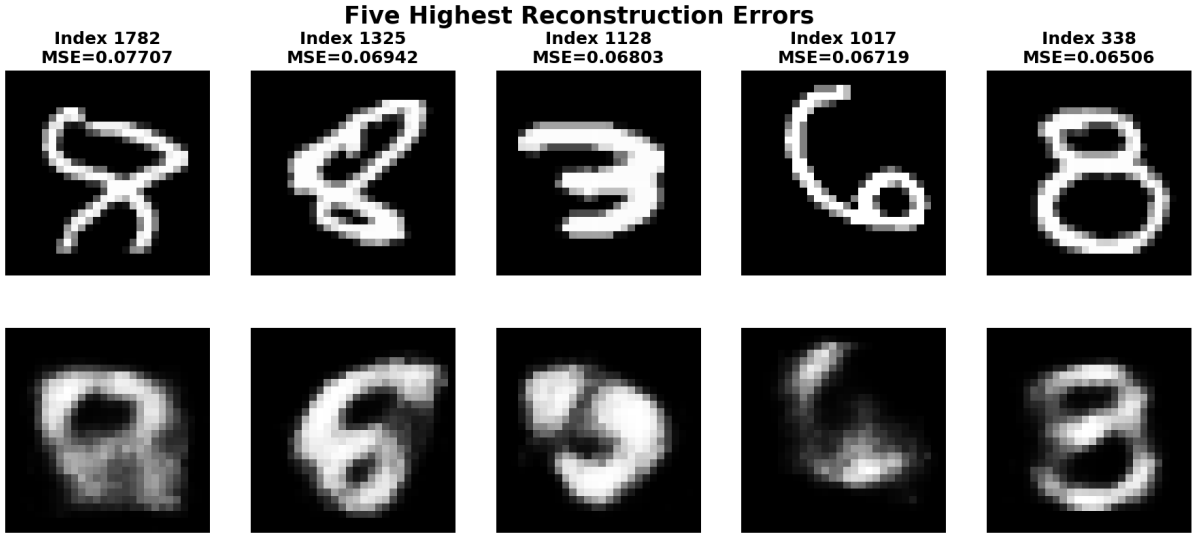


Figure 13: Five test images with the highest reconstruction errors and their corresponding reconstructions.

Table 8: Analysis of the five highest reconstruction-error samples.

Sample	Observation
1	The reconstruction differs from the original mainly in the fine stroke details and local pixel structure.
2	The model preserves the main digit structure, but some smaller or thinner portions of the digit are not reproduced accurately.
3	The reconstructed image is smoother than the original, producing a larger pixel-level difference in some regions.
4	The sample contains image characteristics that are harder for the compressed latent representation to reproduce accurately.
5	The main shape is retained, but differences in stroke thickness or local structure contribute to the higher reconstruction error.

Interpretation

The high-error samples show that reconstruction error is not based only on whether the digit is still recognizable. A reconstructed image may preserve the overall digit shape but still have a higher pixel-level error due to differences in stroke thickness, small curves, or other local details.

These samples also help show the limitations of the learned latent representation. Since the input image is compressed into a smaller latent representation, some details may not be reproduced exactly during decoding, even though the main structure of the digit is preserved.

24. Consolidated Results

The results obtained from the different autoencoder experiments are summarized in Table 9. The reconstruction metrics were calculated on the test set.

Table 9: Consolidated Results of the Autoencoder Experiments

Model	MSE	MAE	SSIM	Parameters	Time (s)
Fully Connected AE	0.022480	0.059559	0.730615	211040	12.9490
Convolutional AE	0.002697	0.015370	0.973007	74497	19.8459
Denoising CAE	0.003673	0.018712	0.954060	74497	35.9426
VAE ($z = 2$)	0.044133	0.103115	0.491307	485957	30.0672

25. Required Inferences

Effect of Latent Dimension on Reconstruction

The latent-dimension experiment was performed using latent dimensions $z \in \{2, 8, 16, 32\}$. The obtained results are shown in Table 10.

Table 10: Effect of latent dimension on FC-AE reconstruction.

Latent Dimension	MSE	SSIM
2	0.057100	0.303824
8	0.033302	0.623377
16	0.024675	0.705872
32	0.021498	0.743559

As the latent dimension increases from 2 to 32, the MSE decreases from 0.057100 to 0.021498, while SSIM increases from 0.303824 to 0.743559. This shows that a very small latent space loses more information during compression. Increasing the latent dimension allows more information from the input image to be retained, resulting in better reconstruction quality.

The result also shows the trade-off between compression and reconstruction. A smaller latent representation gives stronger compression but loses more image information, while a larger representation preserves more information and improves reconstruction quality.

1. Fully Connected AE versus Convolutional AE

The FC-AE obtained an MSE of 0.022480, MAE of 0.059559, and SSIM of 0.730615. In comparison, the CAE obtained a lower MSE of 0.002697, MAE of 0.015370, and a higher SSIM of 0.973007.

The CAE therefore produced better reconstruction results on the test set. This difference is related to how the two models process the image. The FC-AE first flattens the image into a vector, while the CAE works directly with the two-dimensional image and preserves local spatial relationships through convolutional operations.

2. Effect of Gaussian Noise Level

The denoising experiment showed that increasing the Gaussian noise level increased the reconstruction error. For $\sigma = 0.1$, the MSE, MAE and SSIM were 0.002608, 0.015477 and 0.968226, respectively. For $\sigma = 0.3$, these values changed to 0.005617, 0.023899 and 0.926540.

Thus, stronger corruption makes the reconstruction task more difficult. The increase in MSE and MAE and the decrease in SSIM show that the difference between the clean and reconstructed images becomes larger as the noise level increases.

3. Ability of the Denoising Autoencoder to Recover Image Structure

The clean, noisy and denoised images show that the denoising autoencoder can recover the main structure of the digit from a corrupted input. The denoised images are generally clearer than the noisy inputs, although some fine details may be slightly smoothed during reconstruction.

The quantitative results support this observation. At $\sigma = 0.2$, the denoising CAE obtained an MSE of 0.003673, MAE of 0.018712 and SSIM of 0.954060. This shows that the reconstructed images retain a high degree of structural similarity with the corresponding clean images.

4. Characteristics of the VAE Latent Distribution

The VAE does not represent an input using only one fixed latent vector. Instead, the encoder learns the parameters μ and $\log \sigma^2$ of a latent probability distribution, and a latent vector is sampled using the reparameterization trick.

The two-dimensional latent representation provides a continuous space in which different image representations can be located. Points can also be sampled from this latent space and passed through the decoder to generate new images.

5. Structure of the Two-Dimensional VAE Latent Space

The VAE latent-space plot shows the distribution of test images using the two latent variables z_1 and z_2 . Images with similar digit patterns tend to appear in nearby regions, while some overlap can be seen between different digit groups.

This overlap is expected because the VAE is not trained using digit labels. Instead, it learns a probabilistic latent representation that can be used for both reconstruction and generation, rather than explicitly separating the ten digit classes.

6. Quality and Diversity of Generated Samples

Random latent vectors sampled from $\mathcal{N}(0, I)$ were passed through the VAE decoder to generate new images. The generated samples contain digit-like patterns and show variation because different latent vectors are used as decoder inputs.

Some generated samples may appear less clear or ambiguous. This can happen because random sampling may produce latent points that do not correspond to a clearly defined digit region. Overall, the experiment shows that the VAE can generate new outputs instead of only reconstructing previously seen input images.

7. Smoothness of Latent-Space Interpolation

Latent-space interpolation was performed using

$$z(\alpha) = (1 - \alpha)z_A + \alpha z_B, \quad \alpha \in [0, 1]. \quad (19)$$

The decoded images along the interpolation path demonstrate how the generated output changes when the latent representation is moved between two selected points. The intermediate samples provide a way to observe the continuous nature of the learned latent space.

8. Relationship Between Reconstruction Error and Visual Quality

The reconstruction-error distribution shows that different test images have different reconstruction errors. Images with lower errors are closer to the original at the pixel level, while high-error samples have larger differences between the original and reconstructed images.

However, reconstruction error cannot be judged only by whether the digit remains recognizable. A reconstructed digit may preserve its overall shape while still having a higher pixel-level error due to differences in stroke thickness, curves, or other local details.

9. Deterministic versus Probabilistic Latent Representations

The fully connected, convolutional and denoising autoencoders use a deterministic latent representation, where each input is mapped to a fixed latent representation.

In contrast, the VAE learns a probability distribution for the latent representation. The encoder produces μ and $\log \sigma^2$, and a latent vector is sampled from this distribution. This probabilistic representation allows new latent points to be sampled and decoded into new images, and also supports interpolation within the learned latent space.

26. Additional Latent-Dimension Study

To study the effect of the bottleneck size on reconstruction quality, the fully connected autoencoder was trained using four different latent dimensions:

$$d_z \in \{2, 8, 16, 32\}. \quad (20)$$

The remaining architecture and training configuration were kept the same. The reconstruction performance was evaluated using Mean Squared Error (MSE) and Structural Similarity Index (SSIM).

Table 11: Effect of latent dimension on fully connected autoencoder reconstruction.

Latent Dimension	MSE	SSIM	Parameters
2	0.057100	0.303824	—
8	0.033302	0.623377	—
16	0.024675	0.705872	—
32	0.021498	0.743559	—

Note: The executed result table available from the experiment contains MSE and SSIM for this study. The parameter-count values were not recorded in that result table, so they are not fabricated here.

26.1 Latent Dimension versus Reconstruction MSE

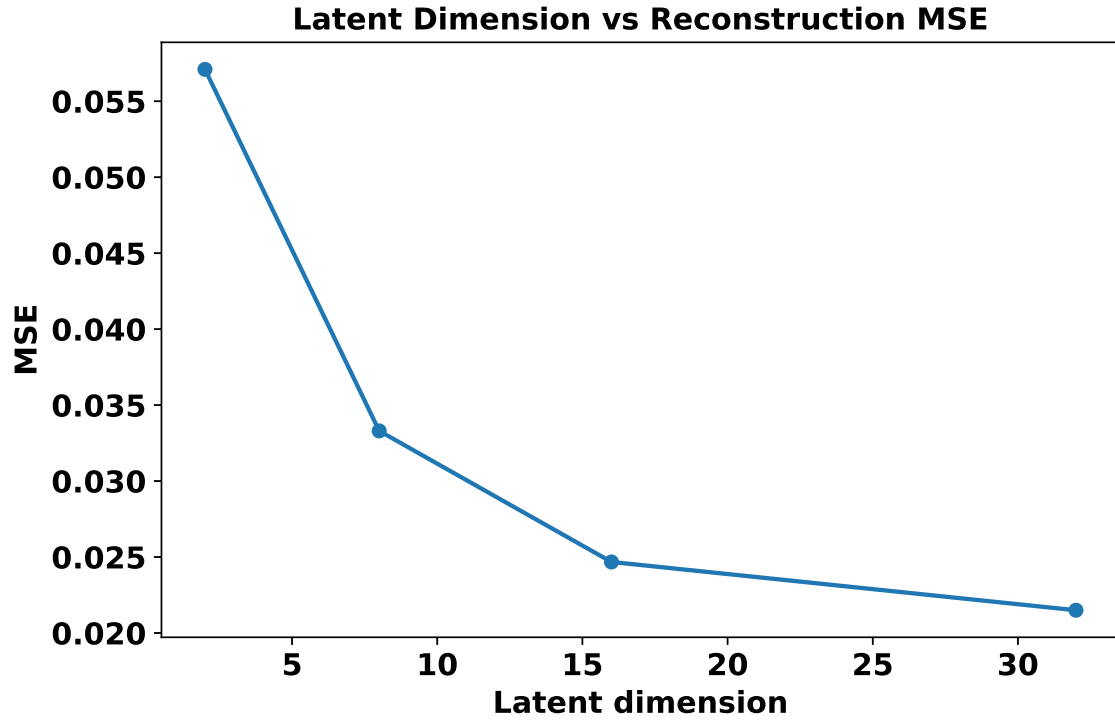


Figure 14: Effect of latent dimension on reconstruction MSE.

Inference

The results show that increasing the latent dimension improved the reconstruction quality in this experiment. The MSE decreased from 0.057100 for a latent dimension of 2 to 0.021498 for a latent dimension of 32, while SSIM increased from 0.303824 to 0.743559.

This indicates that a larger bottleneck allowed the autoencoder to preserve more information about the original image. With a very small latent dimension such as 2, the strong compression constraint causes more information to be lost, resulting in larger reconstruction errors.

As the latent dimension increases, more information can be stored in the latent representation, making the reconstructed images closer to the originals. However, this also reduces the amount of compression. Therefore, the latent dimension represents a trade-off between **compression** and **reconstruction quality**.

In this experiment, the latent dimension of 32 produced the lowest MSE and highest SSIM among the four tested configurations.

27. Discussion Questions

1. What is an autoencoder?

An autoencoder is a neural network that learns to reconstruct its input. It consists of an encoder, a latent representation and a decoder.

$$x \rightarrow \text{Encoder} \rightarrow z \rightarrow \text{Decoder} \rightarrow \hat{x} \quad (21)$$

The model is trained so that the reconstructed image $\hat{x}$ is as close as possible to the original input x .

2. What is the purpose of the encoder?

The encoder transforms the input image into a compact latent representation. It extracts the important information required by the decoder for reconstruction.

3. What is the purpose of the decoder?

The decoder takes the latent representation and reconstructs the original input. Its output is denoted by $\hat{x}$.

4. What is a latent representation?

A latent representation is a compressed representation of the input learned by the encoder. It contains important features of the input in a lower-dimensional space.

5. Why is the latent representation usually smaller than the input?

A smaller latent representation forces the network to learn the important features rather than simply copying the input. It also reduces the amount of information that must be stored in the bottleneck.

6. What happens when the latent dimension is too small?

If the latent dimension is too small, the model cannot retain sufficient information about the input. This generally leads to larger reconstruction errors and loss of fine details.

7. What happens when the latent dimension is increased?

Increasing the latent dimension provides more capacity for storing information. In the FC-AE latent-dimension experiment, increasing the dimension from 2 to 32 reduced MSE and increased SSIM.

However, a larger latent representation also reduces the amount of compression and can increase the number of trainable parameters.

8. Why can a convolutional autoencoder preserve spatial information more effectively?

Convolutional layers operate on local regions of an image and preserve spatial relationships between neighboring pixels. This makes them suitable for extracting image features such as edges, curves and local shapes.

9. Differentiate an autoencoder and a denoising autoencoder.

A conventional autoencoder receives a clean image and attempts to reconstruct the same image:

$$x \rightarrow \hat{x}. \quad (22)$$

A denoising autoencoder receives a corrupted image but uses the original clean image as the target:

$$\tilde{x} \rightarrow \hat{x}, \quad \text{target} = x. \quad (23)$$

10. Why is the clean image used as the target in denoising?

The clean image provides the desired output. The network therefore learns to remove the corruption while preserving the useful structure of the original image.

11. What happens when the noise level is increased?

Increasing the noise level makes the input more different from the clean image. In the experiment, increasing Gaussian noise caused MSE and MAE to increase while SSIM decreased.

12. Why are MSE, MAE and SSIM useful for image reconstruction?

MSE measures the average squared pixel difference, while MAE measures the average absolute pixel difference. SSIM evaluates structural similarity between the images.

Using all three metrics gives a broader view of reconstruction quality than relying on a single metric.

13. What is a Variational Autoencoder?

A Variational Autoencoder is a generative autoencoder that learns a probability distribution over the latent representation instead of learning only a deterministic latent vector.

14. How does a VAE differ from a conventional autoencoder?

A conventional autoencoder maps an input to a deterministic latent vector:

$$z = f(x). \quad (24)$$

A VAE learns the parameters of a latent probability distribution:

$$q_{\phi}(z|x) = \mathcal{N}(\mu(x), \text{diag}(\sigma^2(x))). \quad (25)$$

The probabilistic representation allows the VAE to generate new samples from the learned latent space.

15. Why does a VAE learn a distribution instead of a single latent vector?

Learning a distribution creates a structured and continuous latent space. It allows the model to sample different latent vectors and generate different outputs.

16. What is the reparameterization trick?

The reparameterization trick expresses the sampled latent vector as

$$z = \mu + \sigma \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I). \quad (26)$$

This separates the random sampling operation from the learnable parameters μ and σ , allowing gradients to pass through the sampling operation during training.

17. Why is ϵ sampled from a standard normal distribution?

The standard normal distribution provides a simple fixed source of randomness. The encoder then transforms this noise using μ and σ to obtain samples from the learned latent distribution.

18. What is the role of KL divergence in the VAE loss?

KL divergence measures the difference between the learned latent distribution and the chosen prior distribution. It regularizes the latent space and encourages it to remain close to the prior.

The KL term used in the experiment is

$$L_{KL} = -\frac{1}{2} \sum (1 + \log \sigma^2 - \mu^2 - \sigma^2). \quad (27)$$

19. What is the prior distribution used in the VAE?

The prior distribution used in the experiment is the standard normal distribution:

$$p(z) = \mathcal{N}(0, I). \quad (28)$$

20. Why is a two-dimensional latent space useful for visualization?

A two-dimensional latent space can be plotted directly using two axes. This allows the distribution of encoded samples and the organization of different digit regions to be visually inspected.

21. What does latent-space interpolation demonstrate?

Latent-space interpolation demonstrates how the decoder output changes as the latent representation moves between two points.

The interpolation used in this experiment is

$$z(\alpha) = (1 - \alpha)z_A + \alpha z_B. \quad (29)$$

A smooth sequence of decoded images indicates that meaningful changes can be represented continuously in the latent space.

22. Why can a VAE generate new images?

A VAE can generate new images because its latent space is probabilistic and regularized towards a known prior. A latent vector can be sampled from the prior and passed through the decoder:

$$z \sim \mathcal{N}(0, I) \rightarrow \text{Decoder} \rightarrow \text{Generated Image}. \quad (30)$$

23. Why might some VAE-generated images be unclear?

A randomly sampled latent vector may fall in a region where the decoder has learned a weaker representation. In addition, the VAE balances reconstruction and latent-space regularization, which can result in smoother rather than perfectly sharp outputs.

24. What is the difference between reconstruction and generation?

Reconstruction starts with an existing image and attempts to reproduce that same image:

$$x \rightarrow z \rightarrow \hat{x}. \quad (31)$$

Generation starts by sampling a latent vector and then producing an image that was not supplied as the decoder's original input:

$$z \sim p(z) \rightarrow \hat{x}. \quad (32)$$

25. Why should test images remain unseen during training?

The test set should remain unseen so that the measured reconstruction performance represents the model's behavior on data that was not used for learning. This provides a more meaningful evaluation of the trained model.

28. Additional Exercise

Additional Exercise 1: CAE Latent-Dimension Study

The convolutional autoencoder was trained using four different latent dimensions: 4, 8, 16 and 32. The encoder and decoder architecture was kept the same while the size of the bottleneck representation was changed.

Table 12: CAE performance for different latent dimensions.

Latent Dimension	MSE	MAE	SSIM	Parameters	Time (s)
4	0.047822	0.107770	0.449029	102725	21.17
8	0.027910	0.068708	0.698400	127817	19.95
16	0.040034	0.093546	0.549730	178001	20.48
32	0.011560	0.036149	0.875795	278369	21.25

The results show that the latent dimension affects the reconstruction quality of the CAE. In this experiment, a latent dimension of 32 produced an MSE of 0.011560 and an SSIM of 0.875795. The number of parameters also increased as the latent dimension increased.

Additional Exercise 2: Gaussian Noise versus Salt-and-Pepper Noise

Two different corruption types were compared using the same denoising architecture and training configuration. Gaussian noise was evaluated at $\sigma = 0.1, 0.2, 0.3$, while salt-and-pepper noise was evaluated at $p = 0.05, 0.10, 0.20$.

Table 13: Comparison of Gaussian and salt-and-pepper noise.

Noise Type	Level	MSE	MAE	SSIM
Gaussian	0.10	0.002608	0.015477	0.968226
Gaussian	0.20	0.003673	0.018712	0.954060
Gaussian	0.30	0.005617	0.023899	0.926540
Salt-and-Pepper	0.05	0.002882	0.015879	0.969445
Salt-and-Pepper	0.10	0.003533	0.017576	0.961389
Salt-and-Pepper	0.20	0.005555	0.022419	0.936122

For both noise types, increasing the corruption level resulted in higher MSE and MAE and lower SSIM. At the highest tested level, Gaussian noise with $\sigma = 0.3$ produced an MSE of 0.005617 and an SSIM of 0.926540, while salt-and-pepper noise with $p = 0.20$ produced an MSE of 0.005555 and an SSIM of 0.936122.

This shows that stronger corruption makes reconstruction more difficult, with a greater difference between the clean and reconstructed images.

Additional Exercise 3: SSIM at Two Noise Levels

Two noise levels from each corruption family were selected and compared using Structural Similarity Index (SSIM).

Table 14: SSIM comparison at two noise levels.

Noise Type	Noise Level	SSIM
Gaussian	0.10	0.968226
Gaussian	0.30	0.926540
Salt-and-Pepper	0.05	0.969445
Salt-and-Pepper	0.20	0.936122

For Gaussian noise, SSIM decreased from 0.968226 at $\sigma = 0.1$ to 0.926540 at $\sigma = 0.3$. For salt-and-pepper noise, SSIM decreased from 0.969445 at $p = 0.05$ to 0.936122 at $p = 0.20$. This shows that increasing the corruption level reduces the structural similarity of the reconstructed images.

Additional Exercise 4: UpSampling Decoder versus Transposed-Convolution Decoder

Two decoder architectures were compared while keeping the encoder-side structure unchanged. The first decoder used UpSampling2D followed by Conv2D, while the second decoder used Conv2DTranspose.

Table 15: Comparison of CAE decoder architectures.

Decoder	MSE	MAE	SSIM	Parameters	Time (s)
UpSampling2D + Conv2D	0.002697	0.015370	0.973007	74497	19.85
Conv2DTranspose	0.002343	0.014190	0.976461	111425	20.17

The UpSampling2D + Conv2D decoder produced an MSE of 0.002697, MAE of 0.015370 and SSIM of 0.973007. The Conv2DTranspose decoder produced a lower MSE of 0.002343, MAE of 0.014190 and a higher SSIM of 0.976461.

The Conv2DTranspose configuration used 111425 parameters, compared with 74497 parameters for the UpSampling2D + Conv2D configuration. This shows that the Conv2DTranspose decoder achieved slightly better reconstruction metrics in this experiment, but with a larger number of parameters.

Additional Exercise 5: VAE Latent Dimension 2 versus 8

The VAE was trained using latent dimensions of 2 and 8. Reconstruction metrics and the individual VAE loss components were evaluated on the test set.

Table 16: Comparison of VAE latent dimensions.

z	MSE	MAE	SSIM	Recon. Loss	KL Loss	Total Loss	Parameters
2	0.045075	0.102497	0.498454	150.910431	6.160273	157.070694	485957
8	0.020555	0.053061	0.778949	96.260437	17.098898	113.359329	506321

Increasing the latent dimension from 2 to 8 reduced the test MSE from 0.045075 to 0.020555 and increased SSIM from 0.498454 to 0.778949. The reconstruction loss also decreased from 150.910431 to 96.260437, while the KL loss increased from 6.160273 to 17.098898.

This shows that the larger latent space improved reconstruction quality in this experiment, although it also resulted in a higher KL loss.

Additional Exercise 6: Effect of KL-Loss Contribution

The contribution of the KL-divergence term was studied by varying β in the VAE objective:

$$L_{\text{VAE}} = L_{\text{reconstruction}} + \beta L_{\text{KL}}. \quad (33)$$

The values of β considered were 0, 0.5, 1 and 2.

Table 17: Effect of β on VAE reconstruction and KL losses.

β	MSE	MAE	SSIM	Recon. Loss	KL Loss	Total Loss
0.0	0.044551	0.102260	0.494659	150.338715	35.531761	150.338715
0.5	0.045809	0.106804	0.471470	152.950684	6.221042	156.061188
1.0	0.044468	0.104731	0.483875	150.113312	5.751073	155.864365
2.0	0.046730	0.110585	0.458314	155.426880	4.548447	164.523758

The results show that changing β affected the contribution of the KL-divergence term to the VAE objective. The KL loss decreased from 35.531761 at $\beta = 0$ to 4.548447 at $\beta = 2.0$. The reconstruction metrics also changed across the tested values of β , showing the effect of changing the balance between reconstruction and latent-space regularization.

Additional Exercise 7: Larger VAE Generated Sample Grid

The trained two-dimensional VAE was used to generate 100 new samples by randomly sampling latent vectors from the standard normal distribution.

$$z \sim \mathcal{N}(0, I) \quad (34)$$

VAE — 100 Randomly Generated Samples

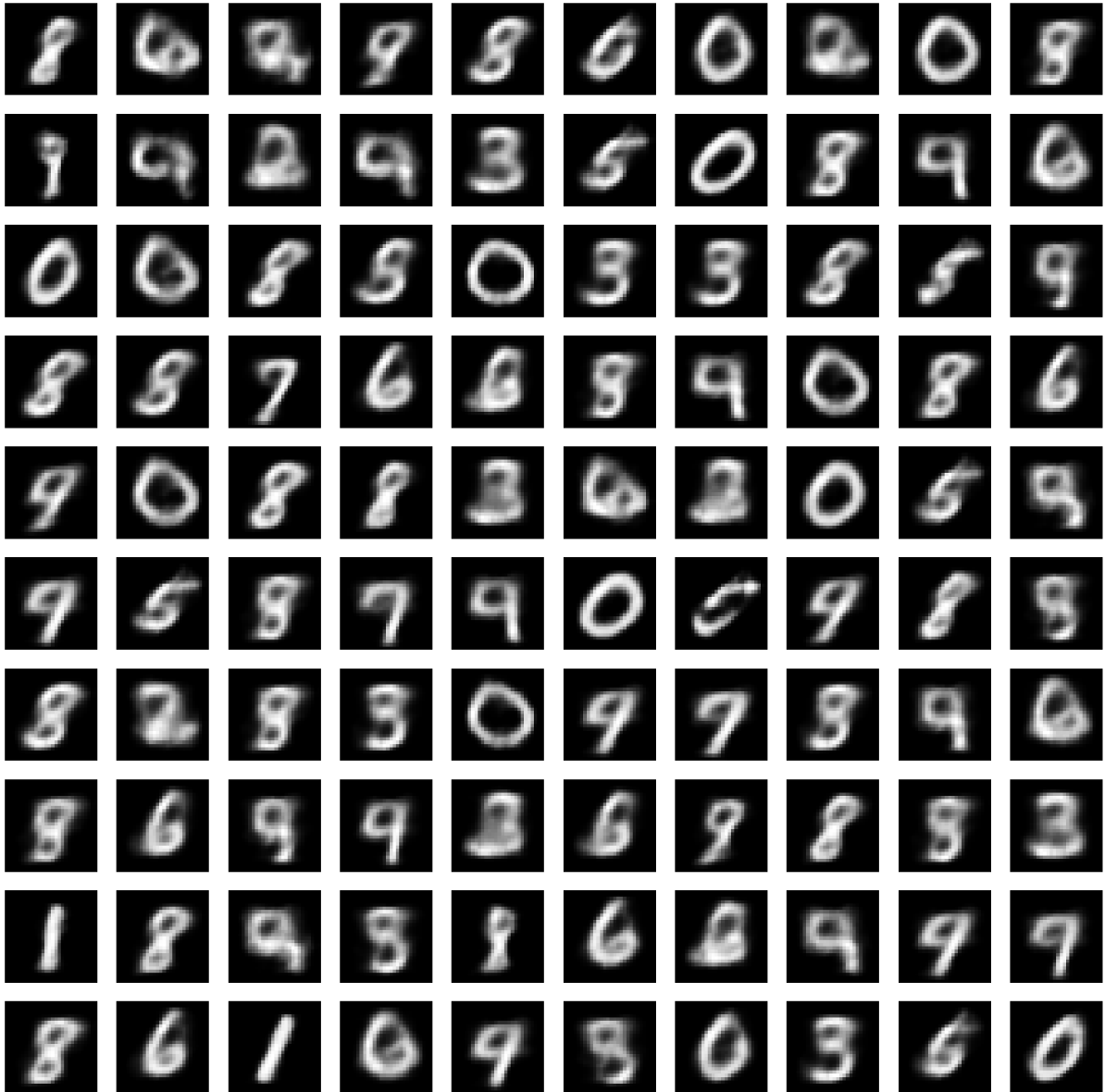


Figure 15: 100 randomly generated samples from the VAE.

The 10×10 generated grid provides a larger set of samples for examining the variation produced by the VAE decoder. The samples are generated from independently sampled latent vectors, rather than being reconstructions of the test images.

Additional Exercise 8: Latent-Region Interpolation

Latent-region interpolation was performed between the mean latent representations of the digit 1 and digit 7 regions.

Let z_A represent the mean latent vector of the digit 1 region and z_B represent the mean latent vector of the digit 7 region. The intermediate latent vectors were calculated using

$$z(\alpha) = (1 - \alpha)z_A + \alpha z_B, \quad \alpha \in [0, 1]. \quad (35)$$

Eleven values of α between 0 and 1 were used to generate the intermediate samples.

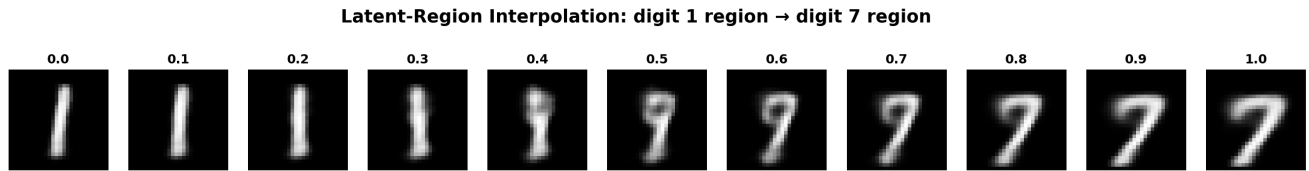


Figure 16: Latent-region interpolation from digit 1 to digit 7.

The sequence of decoded images illustrates how the generated output changes while moving through the latent space from the region associated with digit 1 towards the region associated with digit 7.

29. Expected Outcome

At the end of the experiment, students should be able to demonstrate the complete image-representation and reconstruction pipeline:

Image → Encoder → Latent Representation → Decoder → Reconstructed Image

They should also demonstrate denoising:

Clean Image → Noise → Denoising Autoencoder → Recovered Image

and variational generation:

Image → (μ, σ) → z → Decoder → Generated/Reconstructed Image

The final report should contain both quantitative evaluation and visual analysis. Numerical performance values must be obtained from the student's own execution.

30. References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Diederik P. Kingma and Max Welling, “Auto-Encoding Variational Bayes,” ICLR, 2014.
3. Pascal Vincent et al., “Stacked Denoising Autoencoders: Learning Useful Representations in a Deep Network with a Local Denoising Criterion,” Journal of Machine Learning Research, 2010.
4. Yann LeCun, Corinna Cortes and Christopher J. C. Burges, “MNIST Handwritten Digit Database.”
5. TensorFlow Documentation: <https://www.tensorflow.org>
6. Keras Documentation: <https://keras.io>
7. scikit-image Documentation: <https://scikit-image.org>

GitHub Repository

The complete implementation, Jupyter Notebook, experimental results, plots, and supporting files for this experiment are available in the following GitHub repository:

<https://github.com/Dheekshitha136/Deep-Learning-Lab/tree/main/Deep-Learning-Lab>

The repository contains the source code and experimental resources used for the implementation of the autoencoder experiments presented in this report.